

학생 : 카로프 파루흐 | 학번 : 202444907

Transfer Learning을 사용한 생활 폐기물 분류

Deep Learning 과목 미니 프로젝트

이미지 분류 (영상 데이터 분류) · 10개 클래스

1. 문제 정의

이 프로젝트는 **이미지 분류** 문제를 해결한다. 즉, 재활용 시스템을 위해 생활 폐기물 이미지를 카테고리별로 자동 분류하는 것이 목표이다. 이는 과제에서 제시된 세 가지 주제 중 하나인 이미지 분류에 해당한다.

파라미터	값
문제 유형	다중 클래스 이미지 분류
입력 변수	224×224×3 픽셀 크기로 변환된 RGB 이미지
출력 변수	battery, biological, cardboard, clothes, glass, metal, paper, plastic, shoes, trash 중 하나의 클래스
Metrics	Accuracy와 macro F1

이 문제의 실용적 의미는 분리수거 자동화에 있으며, 이는 현재 중요한 환경적 ESG 분야와 관련이 있다. 주요 방법으로는 **Transfer Learning**을 선택하였다. 이는 ImageNet으로 사전 학습된 모델의 특징을 활용하는 방식이며, 데이터 양이 제한적인 상황에서 industry standard로 사용되는 방법이다.

2. 데이터 설명

외부 데이터셋인 **Garbage Classification V2**를 사용하였다. 이 데이터셋은 Kaggle의 Suman Kunwar가 제공한 데이터셋이다. 데이터셋은 논문 「The Garbage Dataset (GD): A Multi-Class Image Benchmark for Automated Waste Segregation」(2026)에서 설명되었으며, 저자는 EfficientNetV2S architecture에서 95.13% accuracy benchmark를 제시하였다.

특성	값
전체 이미지 수	12,259
클래스 수	10
형식	JPEG, RGB, 256×256 크기 (standardized_256 버전)
train / val / test 분할	70% / 15% / 15%, stratified
train / val / test 예시 수	8593 / 1833 / 1833

클래스 분포와 불균형

데이터셋은 상당히 불균형하다. 가장 큰 클래스와 가장 작은 클래스의 비율은 **4.18**×이다(clothes: 1892, trash: 453). 이 특징은 전처리 방법 선택, 특히 class weights 적용에 직접적인 영향을 주었다.

클래스	개수	비율
clothes	1892	15.4%
glass	1736	14.2%
plastic	1597	13.0%
shoes	1449	11.8%
cardboard	1411	11.5%
paper	1336	10.9%
metal	930	7.6%
battery	756	6.2%
biological	699	5.7%
trash	453	3.7%

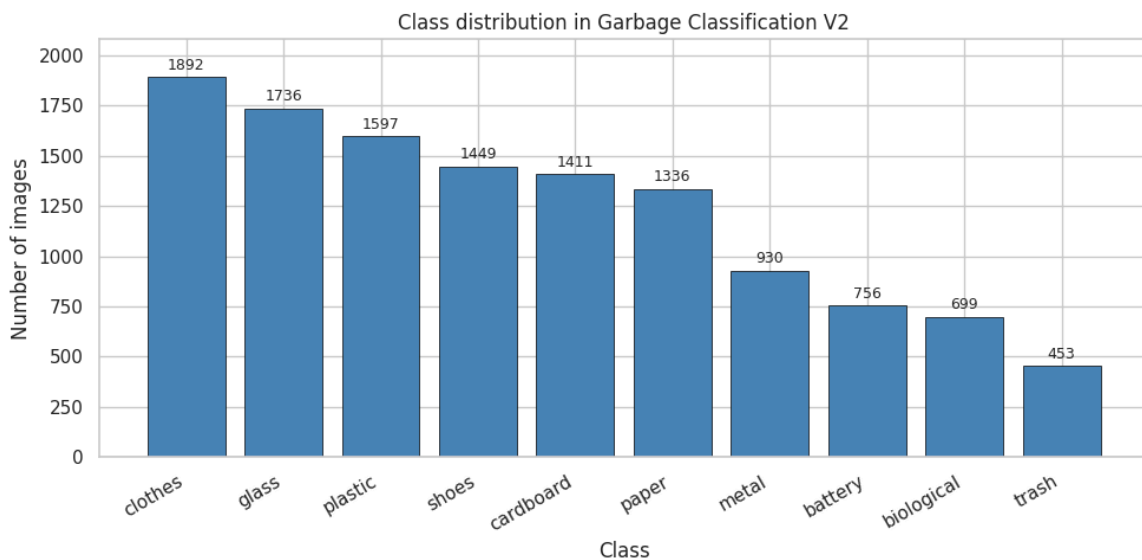


그림 1. 클래스별 이미지 분포

Exploratory Data Analysis (EDA) 결과

각 클래스의 이미지 샘플을 시각적으로 확인한 결과는 다음과 같다.

- **높은 클래스 내부 다양성.** battery(여러 종류의 배터리), biological(빵, 채소, 과일, 피자), trash(마스크, 장갑, 포장재), paper(신문, 종이, 영수증) 클래스는 시각적으로 매우 다양했다.
- **예상되는 confusion pair.** 시각적 유사성을 바탕으로 모델이 혼동할 것으로 예상되는 세 쌍을 미리 설정하였다: paper ↔ cardboard, glass ↔ metal, plastic ↔ glass. 이 세 쌍은 실험 단계에서 모두 확인되었다.

- **Artifacts.** 일부 이미지에는 회색 테두리 영역이 포함되어 있었다. 이는 256×256 크기로 변환하는 과정에서 생긴 결과로 보이며, 따라서 random crop을 augmentation으로 사용하는 근거가 되었다.
- **기술적 일관성.** 확인한 모든 이미지는 256×256, RGB, JPEG 형식임을 확인하였다.

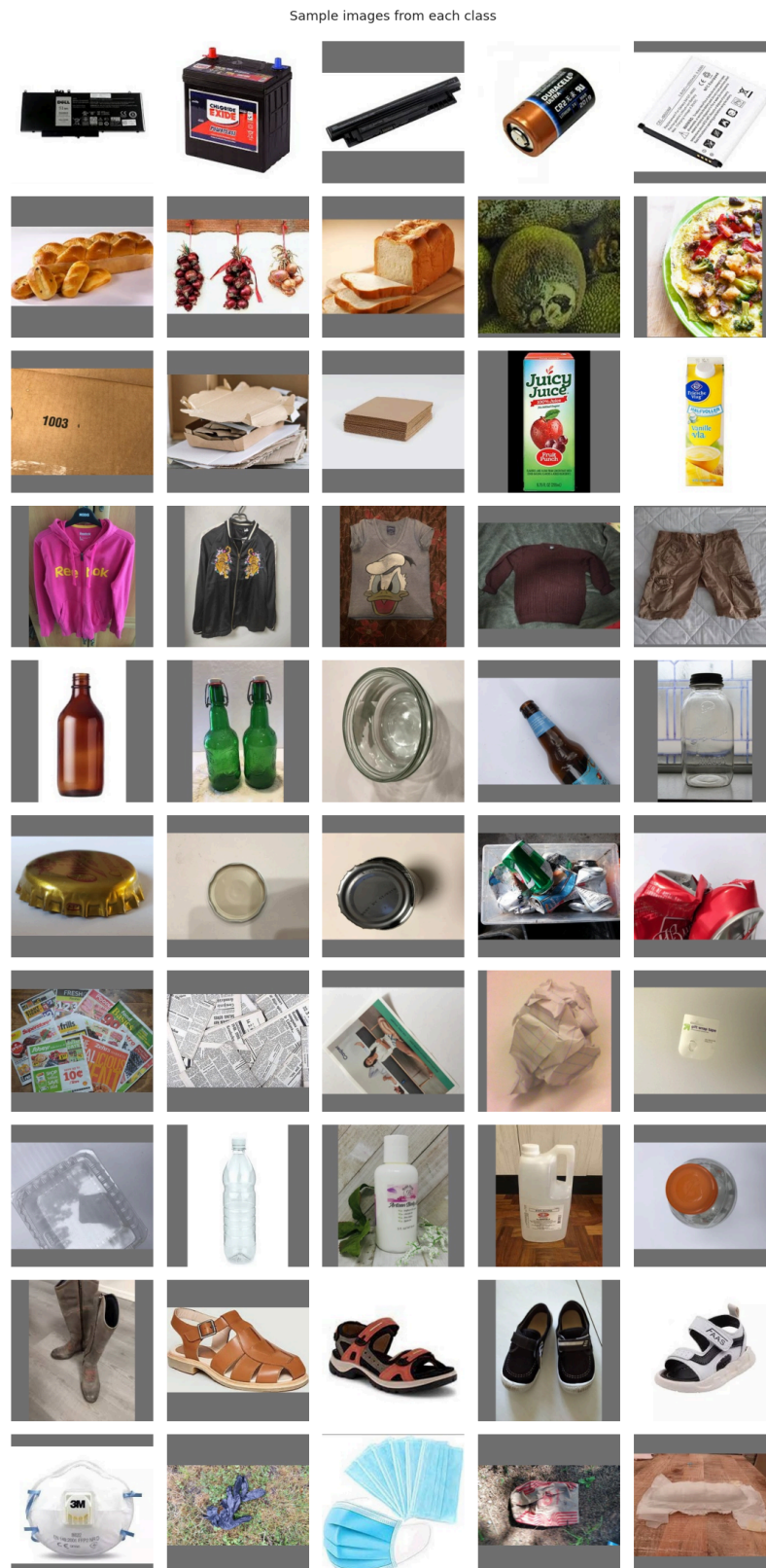


그림 2. 각 클래스의 이미지 예시

3. 데이터 전처리

tf.data 를 기반으로 자체 데이터 처리 pipeline(src/data.py)을 구현하였다.

단계	설명
Stratified split	각 클래스를 독립적으로 70/15/15 비율로 분할하여 모든 subset에서 클래스 비율을 0.1% 미만의 오차로 유지하였다
Augmentation (train only)	224×224 random crop, horizontal flip, brightness 및 contrast 변경($\pm 15\%$)
val / test 처리	metric 재현성을 위해 augmentation 없이 224×224 deterministic center crop 적용
Class weighting	inverse frequency 기준의 weights 적용: trash 2.71×, biological 1.75×, battery 1.62×, clothes 0.65×
Pipeline optimization	GPU 효율적 사용을 위해 parallel loading, shuffling, prefetch 적용

Stratification은 실험적으로 검증하였다. train/val/test 간 클래스 비율 차이는 0.05%를 넘지 않았으며, 이는 불균형 데이터셋에서 신뢰할 수 있는 평가를 위해 중요하다.

Code

본 프로젝트의 모든 코드는 처음부터 직접 작성하였으며, 교수님께서 제공해 주신 template code는 사용하지 않았다. 코드는 재사용 가능한 module로 구성되어 있다:

- src/data.py — data pipeline (stratified split, augmentation, class weights)
- src/model.py — model architectures (baseline CNN, ResNet50, EfficientNetV2S)
- src/train.py — training loop with callbacks
- src/utils.py — metrics logging and visualization

4. Hyperparameter optimization 및 실험

총 5개의 실험을 수행하였다. 과제의 최소 요구 조건은 3개 실험이었지만, 더 명확한 비교를 위해 5개 실험을 진행하였다. 모든 실험은 동일한 training pipeline(src/train.py)을 사용하였고, EarlyStopping, ReduceLROnPlateau, ModelCheckpoint callback을 적용하여 공정한 비교가 가능하도록 하였다. 학습은 Kaggle 환경에서 GPU P100을 사용하여 진행하였다.

번호	모델 / 설정	Test Acc	Macro F1	Epochs	시간
1	Baseline CNN (from scratch, 4 convolution block)	78.61%	0.780	30	5.4분
2	ResNet50, frozen base	92.69%	0.920	17	4.5분
3	ResNet50, fine-tune	93.29%	0.925	26	7.1분
4	EfficientNetV2S, frozen (224px)	92.53%	0.919	20	8.2분
5	ResNet50 fine-tune + class weights	93.51%	0.930	28	7.7분

노란색으로 표시한 설정이 가장 좋은 configuration이다.

실험 1 — Baseline CNN

간단한 convolutional network를 처음부터 학습하였다. 구조는 4개의 Conv→BatchNorm→ReLU→MaxPool block으로 구성되며, 약 390K parameters를 가진다. 결과는 **78.61% accuracy**였다. 중요한 관찰점은 train accuracy(77.7%)가 validation accuracy(78.9%)와 비슷하다는 것이다. 즉, overfitting은 발생하지 않았으며, augmentation이 효과적인 regularizer로 작동하였다. 가장 약한 클래스는 trash($F1 = 0.655$)였고, 가장 강한 클래스는 clothes(0.880), biological(0.874), battery(0.852)였다. EDA에서 예측한 세 가지 confusion pair는 모두 확인되었다.

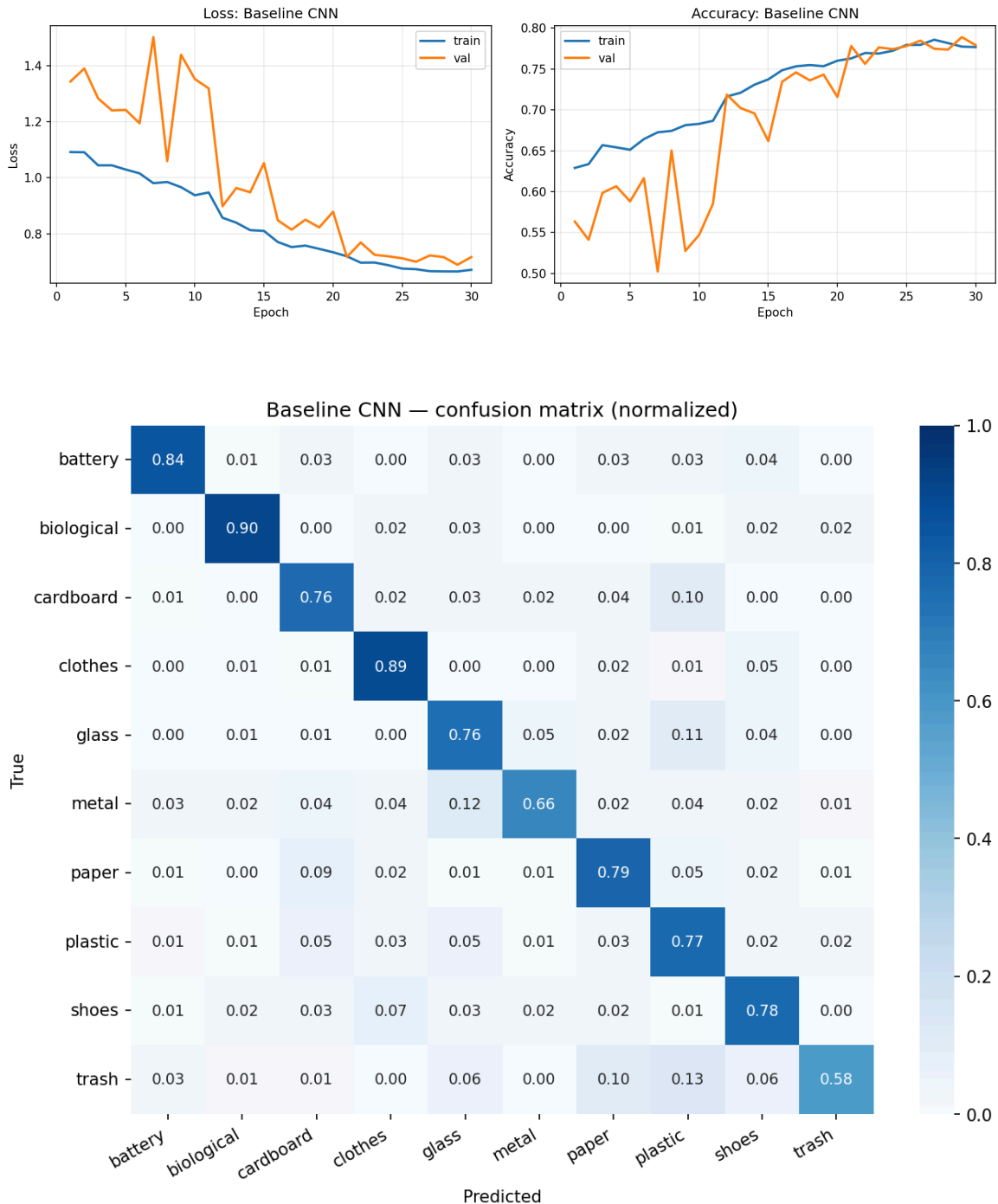


그림 3. Baseline CNN의 learning curves와 confusion matrix

실험 2 — ResNet50 (frozen base)

Transfer Learning을 적용하였다. ImageNet weights를 가진 ResNet50을 feature extractor로 사용하였고, base model은 frozen 상태로 유지하였다. 학습된 것은 classifier 부분뿐이며, 전체 23.6M parameters 중 약 20K parameters만 학습되었다. 결과는 **92.69% accuracy**였으며, baseline 대비 **+14.08 percentage points** 향상되었다. 첫 epoch에서 이미 validation accuracy가 89.7%에 도달했으며, 이는 baseline의 최종 결과보다 높았다. 모든 클래스에서 성능이 향상되었고, 특히 이전에 약했던 클래스에서 가장 큰 향상이 나타났다: metal(+0.24 recall), trash(+0.23), shoes(+0.22).

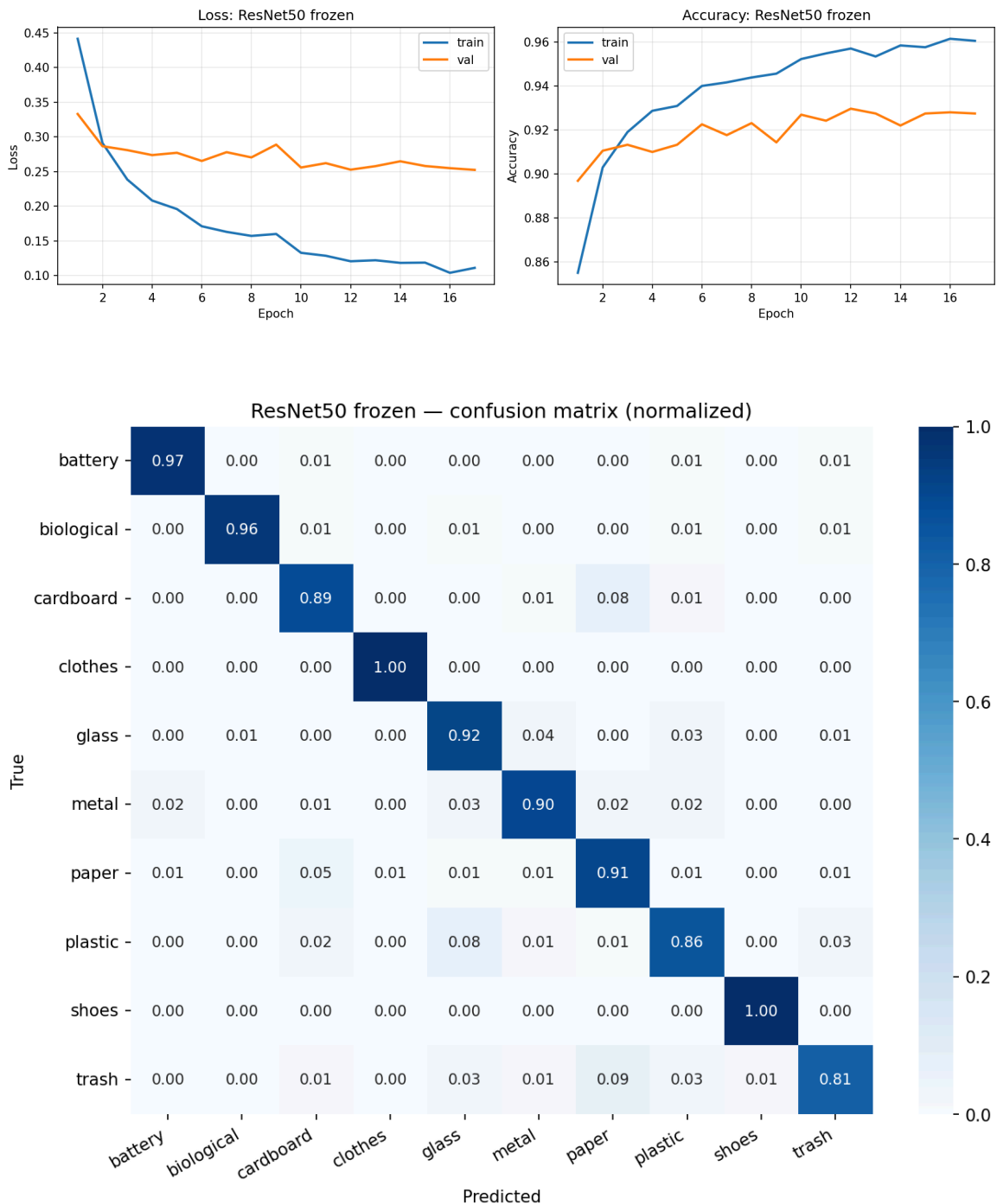


그림 4. ResNet50 frozen의 learning curves와 confusion matrix

실험 3 — ResNet50 (fine-tune)

Two-phase training을 적용하였다. 1단계에서는 frozen base 상태에서 classifier만 학습하였다. 2단계에서는 ResNet50의 마지막 block(conv5_block3)을 unfreeze하고, learning rate를 100배 낮춘 1e-5로 fine-tuning을 진행하였다. ImageNet statistics를 유지하기 위해 BatchNorm layer는 frozen 상태로 유지하였다. 결과는 **93.29% accuracy**였다. 주요 성과는 plastic→glass confusion이 8%에서 5%로 감소한 것이다(상대적 개선 38%). 이는 시각적으로 유사한 클래스에서 fine-tuning이 도움이 된다는 가설을 확인해 준다.

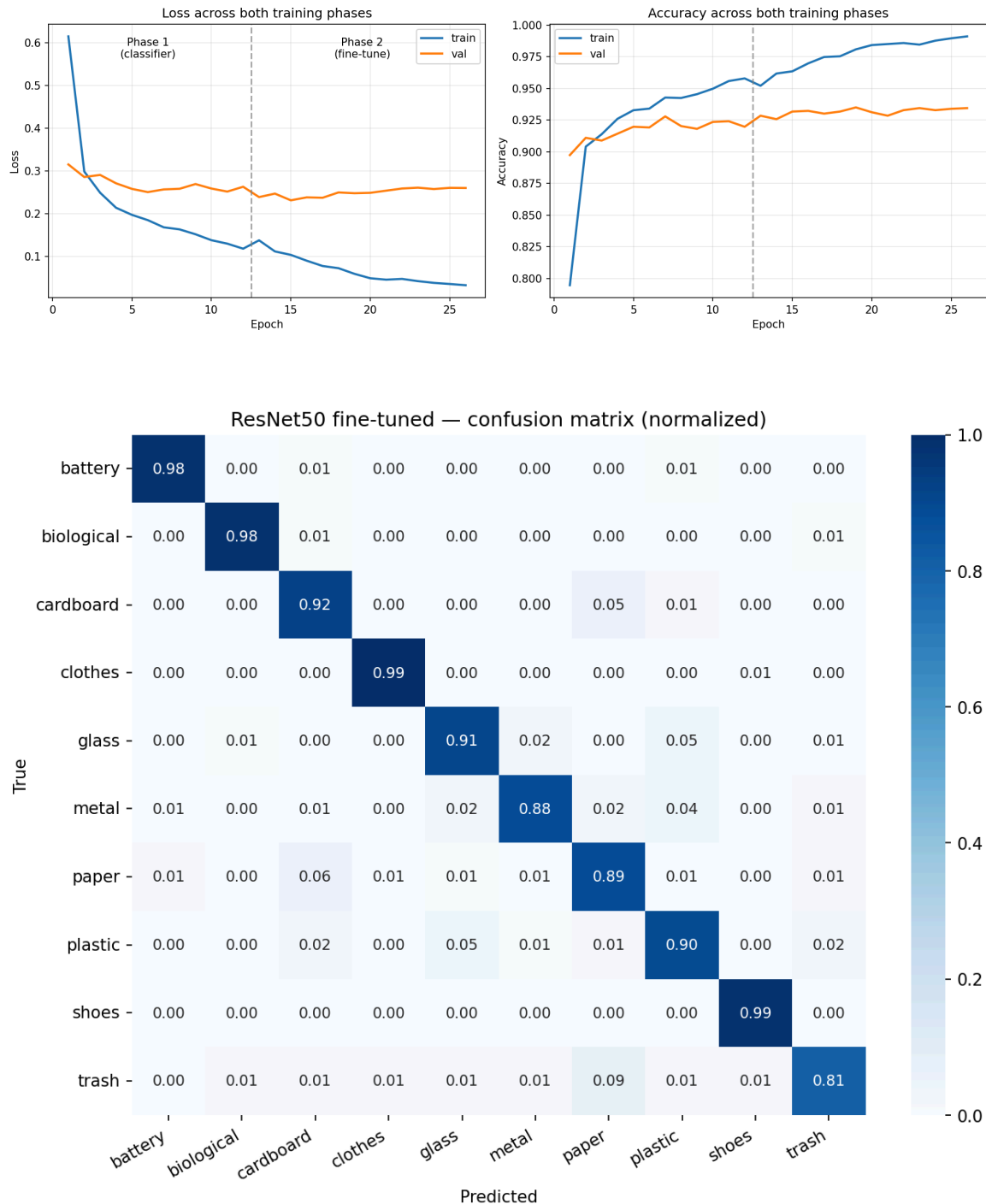


그림 5. ResNet50 fine-tuned의 learning curves(two-phase)와 confusion matrix

실험 4 — EfficientNetV2S

더 현대적인 architecture와의 비교를 위해 EfficientNetV2S를 사용하였다. 결과는 **92.53% accuracy**로, ResNet50보다 약간 낮았다. 이는 방법론적으로 중요한 결론을 제공한다. EfficientNetV2S는 384×384 input을 기준으로 설계된 architecture이며(compound scaling 원리), 본 실험에서는 ResNet50과의 공정한 비교를 위해 224×224 input을 사용하였다. 이 점은 ResNet50보다 낮은 결과와 논문 benchmark(384×384에서 95.13%)와의 2.6 percentage points 차이를 설명해 준다. 하지만 EfficientNetV2S는 가장 낮은 overfitting을 보였으며, train-val gap은 약 1.5%에 불과하였다.

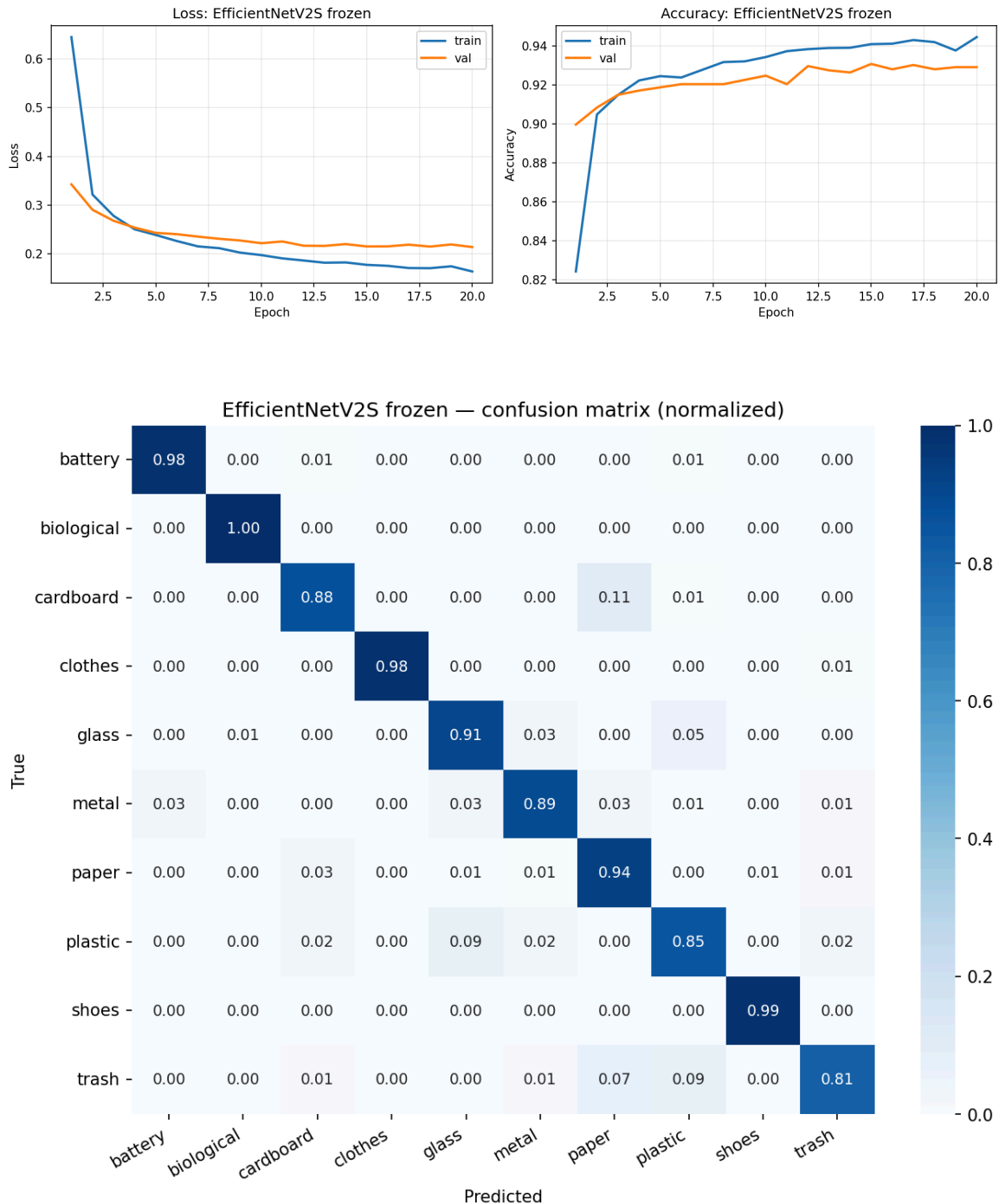


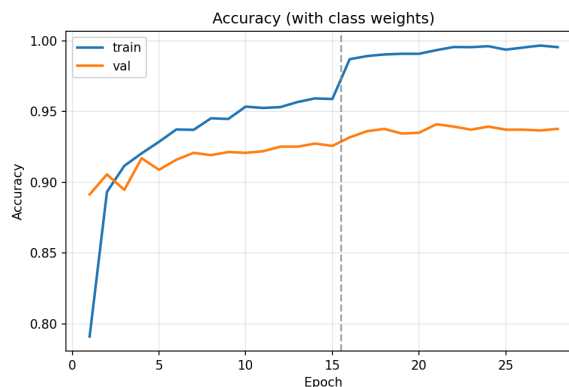
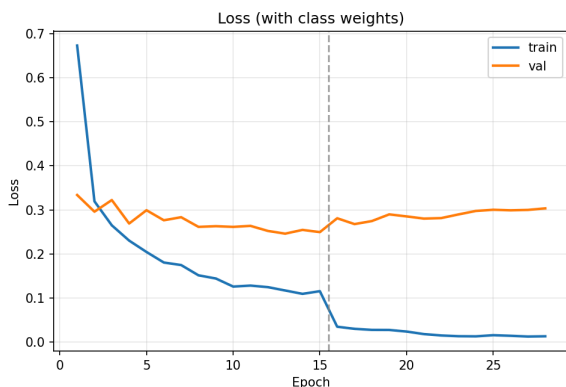
그림 6. EfficientNetV2S의 learning curves와 confusion matrix

실험 5 — ResNet50 fine-tune + class weights (최고 성능 모델)

가장 좋은 모델이었던 실험 3에 inverse frequency 기반 class weights를 추가하였다. 결과는 **93.51% accuracy**로, 프로젝트에서 가장 좋은 성능을 기록하였다. 이 실험의 특징은 overall accuracy와 macro F1이 동시에 향상되었다는 점이다. 일반적으로 class weighting은 overall accuracy를 일부 희생하는 경우가 많지만, 본 실험에서는 moderate weights가 모든 방향에서 성능 향상을 가져왔다.

핵심 성과는 trash 클래스의 “ceiling”을 돌파한 것이다. 실험 2-4에서 trash 클래스의 recall은 architecture와 관계없이 0.81 수준에 머물렀다. 그러나 class weighting을 적용한 결과 recall이 **0.881**까지 상승하였다(+7.5 percentage points). 이는 EDA 단계에서 내린 진단을 확인해 준다. 즉, trash 클래스의 제한 요인은 feature quality나 architecture가 아니라 데이터 불균형이었다.

trash 클래스	weights 없음	weights 적용	Δ
recall	0.806	0.881	+0.075
precision	0.806	0.787	-0.019
F1	0.806	0.831	+0.025



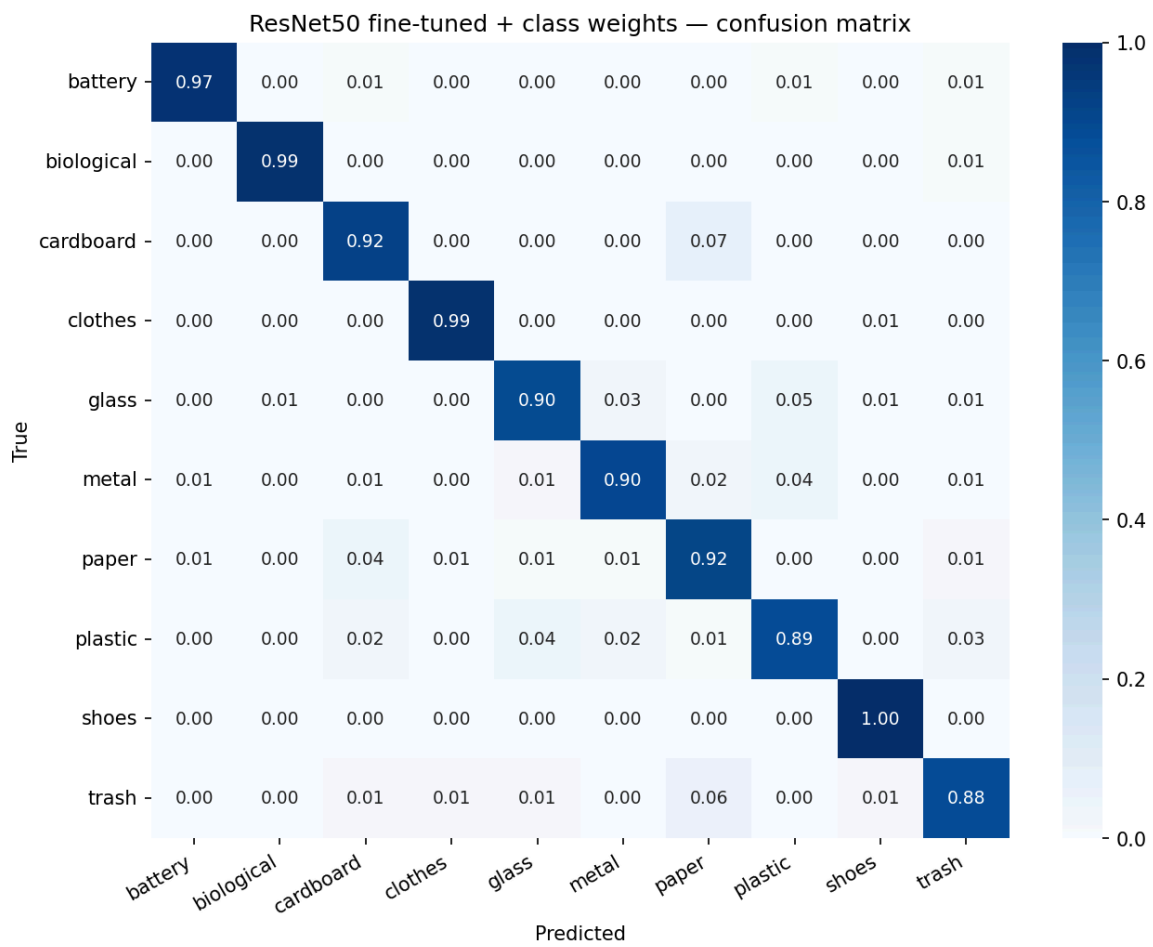


그림 7. 최고 성능 모델의 learning curves와 confusion matrix

5. 결과 요약

최종 hyperparameters (최고 성능 모델)

Hyperparameter	값
Architecture	ResNet50 (ImageNet) + GlobalAveragePooling + Dropout(0.3) + Dense(10)
Training strategy	Two-phase: classifier → 마지막 block fine-tuning
Optimizer	Adam
Learning rate	1e-3 (phase 1) → 1e-5 (phase 2), ReduceLROnPlateau 적용
Batch size	32
Loss function	Sparse Categorical Crossentropy (from logits)
Regularization	Augmentation, Dropout, EarlyStopping
Class weighting	Inverse frequency (trash 2.71× ... clothes 0.65×)
Input size	224×224×3

최종 성능

Metric	값
Test accuracy	93.51%
Test macro F1	0.930
Best class (clothes, F1)	0.989
Worst class (trash, F1)	0.831
Baseline 대비 향상	+14.90 percentage points
논문 benchmark 대비 차이 (95.13%, 384px)	−1.62 percentage points

결론

- **Transfer Learning이 결정적인 기여를 하였다.** From scratch training(78.6%)에서 ResNet50(92.7%)으로 전환하면서 ImageNet semantic features를 활용할 수 있었고, 14 percentage points 이상의 성능 향상을 얻었다.
- **Fine-tuning과 class weighting**을 통해 결과를 93.5%까지 향상시켰으며, EDA 단계에서 진단한 rare class인 trash 문제를 해결하였다.
- **Architecture는 input resolution과 분리해서 볼 수 없다.** EfficientNetV2S는 224px 조건에서 ResNet50을 넘지 못했으며, 이는 compound scaling 이론과 일치한다.
- **EDA의 가치가 확인되었다.** 시각적 분석을 통해 예측한 confusion pair(paper/cardboard, glass/metal, plastic/glass)는 confusion matrix에서 확인되었고, trash 문제의 원인이 feature 문제가 아니라 imbalance라는 진단도 실험을 통해 확인되었다.

향후 개선 방향

- 95.13% benchmark에 더 가까이 도달하기 위해 EfficientNetV2S와 384×384 데이터셋 버전을 사용할 수 있다.
- 독립 데이터셋(예: TrashNet)을 사용하여 Out-of-Distribution generalization을 검증할 수 있다.

Code 및 reproducibility. 전체 code는 재사용 가능한 module(`src/data.py` , `model.py` , `train.py` , `utils.py`) 과 experiment notebooks 형태로 구성하였다.

GitHub: <https://github.com/wayofell/trash-classification>